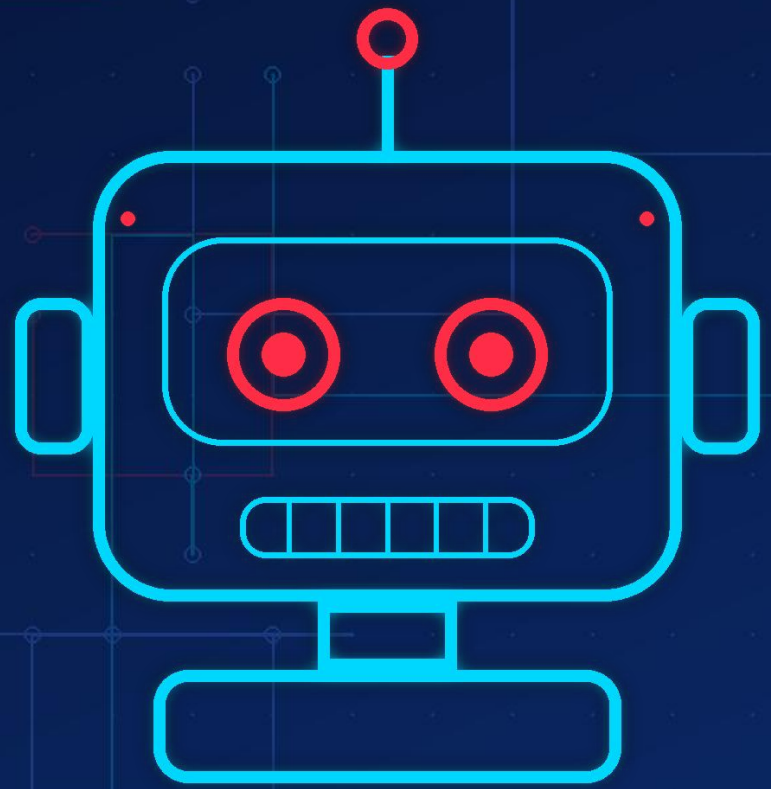
VSP TECHVERSE
An Institute of AI & Robotics

Python (NumPy) for AI & Robotics



A hands-on tour of NumPy arrays — from creating them to filtering them

VSP TECHVERSE
An Institute of AI & Robotics

NumPy HOME

- NumPy = "Numerical Python" — a library for working with arrays
- Powers linear algebra, Fourier transforms and matrices
- The foundation of Python AI, ML and robotics stacks

example.py

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)
print(type(arr))
```

OUTPUT

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
```

AI & ROBOTICS USE

Robot sensor data, images and neural-net weights are all NumPy arrays.

NumPy Intro

- Created in 2005 by Travis Oliphant — open source
- ndarray is much faster than Python lists
- Data sits in one contiguous block of memory; core written in C/C++

example.py

```python
import numpy as np

speeds = np.array([1, 2, 3])

# vectorised: no loop needed
print(speeds * 2)
print(speeds + 10)
```

OUTPUT

```
[2 4 6]
[11 12 13]
```

AI & ROBOTICS USE

Real-time robot control needs fast math on thousands of sensor values.

NumPy Getting Started

- Install with pip (or use Anaconda / Spyder)
- Import with the alias np
- The version is stored in __version__

example.py

```
# terminal
pip install numpy

import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr)
print(np.__version__)
```

OUTPUT

```
[1 2 3 4 5]
2.5.3
```



AI & ROBOTICS USE

Same install on a laptop, Raspberry Pi or Jetson robot controller.

NumPy Creating Arrays

- np.array() turns a list or tuple into an ndarray
- 0-D scalar, 1-D vector, 2-D matrix, 3-D tensor
- ndim shows dimensions; ndmin forces a minimum

example.py

```
import numpy as np

a = np.array(42)
b = np.array([1, 2, 3, 4, 5])
c = np.array([[1, 2, 3], [4, 5, 6]])
d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

print(a.ndim, b.ndim, c.ndim, d.ndim)

arr = np.array([1, 2, 3, 4], ndmin=5)
print(arr, arr.ndim)
```

AI & ROBOTICS USE

1-D: joint angles · 2-D: grayscale image · 3-D: colour camera frame.

OUTPUT

```
0 1 2 3
[[[[[1 2 3 4]]]]] 5
```

NumPy Array Indexing

- Indexes start at 0; -1 is the last element
- 2-D: arr[row, column]
- 3-D: arr[block, row, column]

example.py

```
import numpy as np

arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])

print('2nd element on 1st row: ', arr[0, 1])
print('Last element from 2nd dim: ', arr[1, -1])

arr3 = np.array([[[1, 2, 3], [4, 5, 6]]])
print(arr3[0, 1, 2])
```

OUTPUT

```
2nd element on 1st row:  2
Last element from 2nd dim:  10
6
```

AI & ROBOTICS USE

Pick one sensor reading or one pixel straight from the array.

NumPy Array Slicing

- [start:end] — end is NOT included
- [start:end:step] sets the interval
- 2-D slicing: arr[rows, columns]

example.py

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[1:5])
print(arr[-3:-1])
print(arr[::2])

arr2 = np.array([[1,2,3,4,5], [6,7,8,9,10]])
print(arr2[0:2, 1:4])
```

OUTPUT

```
[2 3 4 5]
[5 6]
[1 3 5 7]
[[2 3 4]
 [7 8 9]]
```

AI & ROBOTICS USE

Crop a region of interest from a camera image or a LiDAR scan.

NumPy Data Types

- i integer · f float · b bool · S/U string · u unsigned
- dtype reveals type; astype() returns a converted copy
- Wrong conversion raises a ValueError

example.py

```
import numpy as np

arr = np.array([1.1, 2.1, 3.1])
print(arr.dtype)

newarr = arr.astype('i')
print(newarr, newarr.dtype)

flags = np.array([1, 0, 3]).astype(bool)
print(flags, flags.dtype)
```

AI & ROBOTICS USE

Choose compact types (int8, float32) to save robot memory.

OUTPUT

```
float64
[1 2 3] int32
[ True False  True] bool
```

NumPy Copy vs View

- copy() is a new array owning its data
- view() only looks at the original data
- Changing a view changes the original!

example.py

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
y = arr.view()

arr[0] = 42
print(arr)
print(x)   # copy unchanged
print(y)   # view changed
```

OUTPUT

```
[42  2  3  4  5]
[1 2 3 4 5]
[42  2  3  4  5]
```

AI & ROBOTICS USE

Avoid accidentally corrupting raw sensor logs while processing.

NumPy Array Shape

- shape returns a tuple: elements per dimension
- (2, 4) = 2 rows, 4 columns
- ndmin adds dimensions — the last value is the element count

example.py

```
import numpy as np

arr = np.array([[1, 2, 3, 4],
                [5, 6, 7, 8]])
print(arr.shape)

arr = np.array([1, 2, 3, 4], ndmin=5)
print(arr)
print('shape of array :', arr.shape)
```

OUTPUT

```
(2, 4)
[[[[[1 2 3 4]]]]]
shape of array : (1, 1, 1, 1, 4)
```

AI & ROBOTICS USE

Always check shapes before feeding data to a neural network.

NumPy Array Reshape

- Change dimensions without changing the data
- Element counts must match (8 → 4×2, not 3×3)
- Use -1 to let NumPy work out one dimension

example.py

```
import numpy as np

arr = np.array([1,2,3,4,5,6,7,8,9,10,11,12])

print(arr.reshape(4, 3))

arr2 = np.array([1, 2, 3, 4, 5, 6, 7, 8])
print(arr2.reshape(2, 4).base)    # a view

print(np.array([[1, 2, 3], [4, 5, 6]]).reshape(-1))
```

AI & ROBOTICS USE

Flatten an image into a feature vector for an ML model.

OUTPUT

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
[1 2 3 4 5 6 7 8]
[1 2 3 4 5 6]
```

NumPy Array Iterating

- A for loop walks the first dimension
- np.nditer() visits every scalar — no nested loops
- np.ndenumerate() gives index and value

example.py

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

for x in np.nditer(arr):
    print(x, end=' ')

print()
for idx, x in np.ndenumerate(arr[0]):
    print(idx, x)
```

AI & ROBOTICS USE

Scan every pixel or reading when vectorising isn't possible.

OUTPUT

```
1 2 3 4 5 6
(0,) 1
(1,) 2
(2,) 3
```

NumPy Array Join

- Joining = merging arrays along an axis (not by key)
- concatenate() and stack() take an axis, default 0
- hstack(), vstack(), dstack() are shortcuts

example.py

```
import numpy as np

arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

print(np.concatenate((arr1, arr2)))
print(np.hstack((arr1, arr2)))
print(np.vstack((arr1, arr2)))
```

OUTPUT

```
[1 2 3 4 5 6]
[1 2 3 4 5 6]
[[1 2 3]
 [4 5 6]]
```



AI & ROBOTICS USE

Merge IMU, camera and LiDAR readings into one input array.

NumPy Array Split

- Splitting is the reverse of joining
- array_split(arr, n) returns a list of arrays
- axis=1 or hsplit() splits by columns

example.py

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

print(np.array_split(arr, 3))
print(np.array_split(arr, 4))

newarr = np.array_split(arr, 3)
print(newarr[0])
```

OUTPUT

```
[array([1, 2]), array([3, 4]), array([5, 6])]
[array([1, 2]), array([3, 4]), array([5]), array([6])]
[1 2]
```

AI & ROBOTICS USE

Split a dataset into training and testing batches.

NumPy Array Search

- where() returns indexes matching a condition
- searchsorted() does binary search on sorted arrays
- side='right' returns the rightmost insert position

example.py

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 4, 4])
print(np.where(arr == 4))

odd = np.array([10, 14, 93, 41, 8, 7])
print(np.where(odd % 2 == 1))

sorted_arr = np.array([6, 7, 8, 9])
print(np.searchsorted(sorted_arr, 7))
print(np.searchsorted(sorted_arr, 7, side='right'))
```

AI & ROBOTICS USE

Locate every obstacle reading beyond a safety threshold.

OUTPUT

```
(array([3, 5, 6]),)
(array([2, 3, 5]),)
1
2
```

NumPy Array Sort

- np.sort() orders numbers, strings or booleans
- Returns a copy — the original stays unchanged
- 2-D arrays are sorted row by row

example.py

```
import numpy as np

print(np.sort(np.array([3, 2, 0, 1])))

names = np.array(['banana', 'cherry', 'apple'])
print(np.sort(names))

arr = np.array([[3, 2, 4], [5, 0, 1]])
print(np.sort(arr))
```

OUTPUT

```
[0 1 2 3]
['apple' 'banana' 'cherry']
[[2 3 4]
 [0 1 5]]
```



AI & ROBOTICS USE

Rank detections by confidence score before choosing an action.

NumPy Array Filter

- Filtering pulls selected elements into a new array
- A boolean index list keeps True, drops False
- Apply the condition directly: arr > 42

example.py

```
import numpy as np

arr = np.array([41, 42, 43, 44])

filter_arr = arr > 42
newarr = arr[filter_arr]

print(filter_arr)
print(newarr)

even = np.array([1, 2, 3, 4, 5, 6, 7])
print(even[even % 2 == 0])
```

AI & ROBOTICS USE

Keep only valid sensor readings and drop the noisy ones.

OUTPUT

```
[False False  True  True]
[43 44]
[2 4 6]
```

NumPy Concatenating Arrays

- np.concatenate() joins arrays along an existing axis
- axis=0 adds rows; axis=1 adds columns
- Arrays must match in every other dimension

example.py

```
import numpy as np

a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

print(np.concatenate((a, b), axis=0))
print(np.concatenate((a, b), axis=1))
```

OUTPUT

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
[[1 2 5 6]
 [3 4 7 8]]
```

AI & ROBOTICS USE

Append new sensor readings to a growing data log.

NumPy Stacking Arrays

- stack() joins arrays along a NEW axis
- vstack() stacks vertically, hstack() horizontally
- dstack() stacks along depth (3rd axis)

example.py

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(np.stack((a, b)))
print(np.vstack((a, b)))
print(np.hstack((a, b)))
print(np.dstack((a, b)).shape)
```

OUTPUT

```
[[1 2 3]
 [4 5 6]]
[[1 2 3]
 [4 5 6]]
[1 2 3 4 5 6]
(1, 3, 2)
```



AI & ROBOTICS USE

Stack R, G, B channels into one colour image array.

NumPy Flattening Arrays

- Flattening turns any array into 1-D
- flatten() always returns a copy
- ravel() returns a view when possible

example.py

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr.flatten())
print(arr.ravel())
print(arr.reshape(-1))
```

OUTPUT

```
[1 2 3 4 5 6]
[1 2 3 4 5 6]
[1 2 3 4 5 6]
```



AI & ROBOTICS USE

Turn a 2-D image into a vector for a classifier.

NumPy Transposing Arrays

- Transposing swaps rows and columns
- Use arr.T or np.transpose(arr)
- Shape (2, 3) becomes (3, 2)

example.py

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr.shape)
print(arr.T)
print(arr.T.shape)
```

OUTPUT

```
(2, 3)
[[1 4]
 [2 5]
 [3 6]]
(3, 2)
```

AI & ROBOTICS USE

Rotate coordinate matrices in robot kinematics.

Filtering with AND, OR, NOT

- Combine conditions with & (AND), | (OR), ~ (NOT)
- Wrap each condition in parentheses
- Python's and / or / not do not work on arrays

example.py

```python
import numpy as np

arr = np.array([10, 25, 40, 55, 70])

print(arr[(arr > 20) & (arr < 60)])   # AND
print(arr[(arr < 20) | (arr > 60)])   # OR
print(arr[~(arr > 40)])               # NOT
```

OUTPUT

```
[25 40 55]
[10 70]
[10 25 40]
```

AI & ROBOTICS USE

Keep readings inside a safe range and drop the rest.

NumPy - Sum

- np.sum() adds all elements
- axis=0 sums down columns, axis=1 across rows
- Also available as arr.sum()

example.py

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

print(np.sum(arr))
print(np.sum(arr, axis=0))
print(np.sum(arr, axis=1))
```

OUTPUT

```
21
[5 7 9]
[ 6 15]
```

AI & ROBOTICS USE

Total energy used or distance travelled by a robot.

NumPy - Mean

- np.mean() returns the average value
- Works along an axis just like sum()
- Sensitive to extreme outliers

example.py

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

print(np.mean(arr))
print(np.mean(arr, axis=0))
print(np.mean(arr, axis=1))
```

OUTPUT

```
3.5
[2.5 3.5 4.5]
[2. 5.]
```

AI & ROBOTICS USE

Smooth noisy sensor values by averaging readings.

NumPy - Median

- np.median() returns the middle value when sorted
- Even count: average of the two middle values
- Not affected by outliers

example.py

```
import numpy as np

arr = np.array([1, 3, 5, 7, 100])

print(np.median(arr))
print(np.mean(arr))    # pulled up by 100

print(np.median(np.array([1, 2, 3, 4])))
```

OUTPUT

```
5.0
23.2
2.5
```

AI & ROBOTICS USE

Reject spikes in distance-sensor data.

NumPy - Min

- np.min() finds the smallest value
- np.argmin() gives its index
- Supports axis for row/column minimums

example.py

```
import numpy as np

arr = np.array([7, 3, 9, 1, 5])

print(np.min(arr))
print(np.argmin(arr))

arr2 = np.array([[4, 2], [8, 6]])
print(np.min(arr2, axis=0))
```

OUTPUT

```
1
3
[4 2]
```

AI & ROBOTICS USE

Find the closest obstacle in a LiDAR scan.

NumPy - Max

- np.max() finds the largest value
- np.argmax() gives its index
- Supports axis for row/column maximums

example.py

```
import numpy as np

arr = np.array([7, 3, 9, 1, 5])

print(np.max(arr))
print(np.argmax(arr))

arr2 = np.array([[4, 2], [8, 6]])
print(np.max(arr2, axis=1))
```

OUTPUT

```
9
2
[4 8]
```

AI & ROBOTICS USE

Pick the class with the highest model confidence.

NumPy - I/O with NumPy

- NumPy can save arrays to disk and load them back
- Text formats: savetxt() / loadtxt() / genfromtxt()
- Binary formats: save() / load() (.npy) and savez() (.npz)

example.py

```python
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

np.save('data.npy', arr)       # write
loaded = np.load('data.npy')    # read

print(loaded)
print(loaded.dtype)
```

OUTPUT

```
[1 2 3 4 5]
int64
```

AI & ROBOTICS USE

Save trained weights and recorded sensor runs between sessions.

NumPy - Reading Data from Files

- np.loadtxt() reads clean numeric text files
- np.genfromtxt() copes with missing values
- delimiter sets the separator; skiprows skips headers

example.py

```
import numpy as np

open('sensor.csv', 'w').write('1,2,3\n4,5,6\n7,8,9')

data = np.loadtxt('sensor.csv', delimiter=',')
print(data)
print(data.shape)
```

OUTPUT

```
[[1. 2. 3.]
 [4. 5. 6.]
 [7. 8. 9.]]
(3, 3)
```



AI & ROBOTICS USE

Load logged IMU or LiDAR readings from a CSV file.

NumPy - Writing Data to Files

- np.savetxt() writes an array as readable text
- fmt controls number format; delimiter the separator
- np.savez() stores several arrays in one file

example.py

```
import numpy as np

arr = np.array([[1.5, 2.5], [3.5, 4.5]])

np.savetxt('out.csv', arr, delimiter=',', fmt='%.1f')

print(open('out.csv').read())
```

OUTPUT

```
1.5,2.5
3.5,4.5
```



AI & ROBOTICS USE

Export predictions so other robot tools can use them.

NumPy - File Formats Supported

- .txt / .csv — human-readable text
- binary .npy (one array) and .npz (many arrays)
- Pick text for sharing, binary for speed and exact types

example.py

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([[4, 5], [6, 7]])

np.savez('bundle.npz', first=a, second=b)

f = np.load('bundle.npz')
print(f.files)
print(f['second'])
```

AI & ROBOTICS USE

Use .npy for fast loading of large camera datasets.

OUTPUT

```
['first', 'second']
[[4 5]
 [6 7]]
```

NumPy - Basics of Date and Time

- datetime64 is NumPy's date and time data type
- Create from ISO strings; units: Y, M, D, h, m, s
- timedelta64 holds differences between dates

example.py

```
import numpy as np

d = np.datetime64('2026-10-11')
print(d)
print(d + 7)    # add 7 days

days = np.array(['2026-10-01', '2026-10-02'],
                dtype='datetime64')
print(days)
print(np.datetime64('2026-12-25') - d)
```

AI & ROBOTICS USE

Time-stamp robot sensor logs and measure intervals.

OUTPUT

```
2026-10-11
2026-10-18
['2026-10-01' '2026-10-02']
75 days
```

NumPy - Handling Missing Data

- Missing data is stored as np.nan (Not a Number)
- nan is a float, so the array becomes float
- Any math with nan returns nan — handle it first

example.py

```
import numpy as np

arr = np.array([1, 2, np.nan, 4, 5])

print(arr)
print(arr.dtype)
print(np.sum(arr))
print(np.nansum(arr))
```

OUTPUT

```
[ 1.  2. nan  4.  5.]
float64
nan
12.0
```



AI & ROBOTICS USE

Sensors drop readings; models need clean arrays.

NumPy - Identifying Missing Values

- np.isnan() flags missing values as True
- np.sum(np.isnan(arr)) counts them
- np.where() locates their positions

example.py

```
import numpy as np

arr = np.array([1, np.nan, 3, np.nan, 5])

mask = np.isnan(arr)
print(mask)
print(np.sum(mask))
print(np.where(mask))
```

OUTPUT

```
[False  True False  True False]
2
(array([1, 3]),)
```

AI & ROBOTICS USE

Find which sensor channels failed during a run.

NumPy - Removing Missing Data

- Filter with the inverse mask: arr[~np.isnan(arr)]
- For 2-D data, drop whole rows that contain nan
- Removing data shortens the array

example.py

```
import numpy as np

arr = np.array([1, np.nan, 3, np.nan, 5])
print(arr[~np.isnan(arr)])

data = np.array([[1, 2], [np.nan, 4], [5, 6]])
print(data[~np.isnan(data).any(axis=1)])
```

OUTPUT

```
[1. 3. 5.]
[[1. 2.]
 [5. 6.]]
```



AI & ROBOTICS USE

Drop corrupted frames before training a model.

NumPy - Imputing Missing Data

- Imputing = replacing nan with a sensible value
- Common choices: mean, median or a constant
- np.nanmean() ignores nan when computing the fill value

example.py

```
import numpy as np

arr = np.array([1, np.nan, 3, np.nan, 5])

fill = np.nanmean(arr)
print(fill)

arr[np.isnan(arr)] = fill
print(arr)
```

OUTPUT

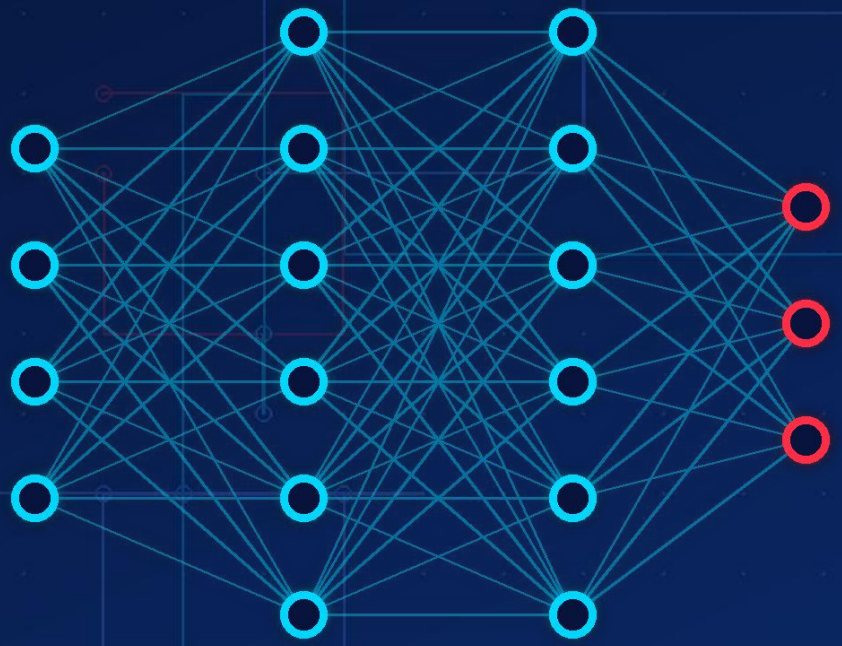
```
3.0
[1. 3. 3. 3. 5.]
```

AI & ROBOTICS USE

Keep every time step by filling sensor gaps with the mean.

Thank You

Keep building smarter machines with Python and NumPy.



VSP TECHVERSE
An Institute of AI & Robotics